🔄 Linked List Reversal & Rotation

Master Node Manipulation with Enhanced Visual Arrows

📋 Table of Contents

1. Introduction - The Pattern
2. How to Recognize These Problems
3. Problem 1: Reverse Nodes in K-Group
4. Problem 2: Reverse Between Positions
5. Problem 3: Rotate List
6. Summary & Pattern Recognition

1 Introduction - The Reversal Pattern

🔧 The Core Concept

Linked List reversal = **rewiring arrows (next pointers)**, NOT swapping values!

🎯 The "Mechanic & Helper" Analogy

Think of rewiring train cars:

🔧 **Mechanic (curr):** Moves forward
🤝 **Helper (prev):** Holds the connection
📦 **Storage (next):** Saves where to go

Rule: Always save next BEFORE breaking connection!

📊 ENHANCED Basic Reversal with Proper Arrows

```
🔍 HOW REVERSAL WORKS - STEP BY STEP

Original List: 1 ──→ 2 ──→ 3 ──→ 4 ──→ null

╔════════════════════════════════════════════╗
║ STEP 0: Initialize Pointers                ║
╚════════════════════════════════════════════╝

prev = null
curr = 1
next = ?

      prev     curr
       │        │
       ↓        ↓
      null    ┌───┐    ┌───┐    ┌───┐    ┌───┐
              │ 1 │───→│ 2 │───→│ 3 │───→│ 4 │───→ null
              └───┘    └───┘    └───┘    └───┘

╔════════════════════════════════════════════╗
║ STEP 1: Save Next, Reverse Arrow           ║
╚════════════════════════════════════════════╝

1. next = curr.next     (Save: next = 2)
2. curr.next = prev     (Reverse: 1 ──→ null)
3. prev = curr          (Move prev to 1)
4. curr = next          (Move curr to 2)

              prev          curr
               │             │
               ↓             ↓
      null ←──┌───┐    ┌───┐    ┌───┐    ┌───┐
              │ 1 │    │ 2 │───→│ 3 │───→│ 4 │───→ null
              └───┘    └───┘    └───┘    └───┘
                         ↑
                        next

╔════════════════════════════════════════════╗
║ STEP 2: Repeat Process                     ║
╚════════════════════════════════════════════╝

1. next = curr.next     (Save: next = 3)
2. curr.next = prev     (Reverse: 2 ──→ 1)
3. prev = curr          (Move prev to 2)
4. curr = next          (Move curr to 3)

                       prev          curr
                        │             │
                        ↓             ↓
      null ←──┌───┐←──┌───┐    ┌───┐    ┌───┐
              │ 1 │    │ 2 │    │ 3 │───→│ 4 │───→ null
              └───┘    └───┘    └───┘    └───┘
                                  ↑
                                 next

╔════════════════════════════════════════════╗
║ STEP 3: Continue...                        ║
╚════════════════════════════════════════════╝

                                prev          curr
                                 │             │
                                 ↓             ↓
      null ←──┌───┐←──┌───┐←──┌───┐    ┌───┐
              │ 1 │    │ 2 │    │ 3 │    │ 4 │───→ null
              └───┘    └───┘    └───┘    └───┘

╔════════════════════════════════════════════╗
║ STEP 4: Final State                        ║
╚════════════════════════════════════════════╝

                                         prev    curr
                                          │       │
                                          ↓       ↓
      null ←──┌───┐←──┌───┐←──┌───┐←──┌───┐    null
              │ 1 │    │ 2 │    │ 3 │    │ 4 │
              └───┘    └───┘    └───┘    └───┘
                                          ↑
                                      NEW HEAD!

✅ FINAL RESULT: prev is the new head
Reversed List: 4 ──→ 3 ──→ 2 ──→ 1 ──→ null
```

💡 Key Insights

- ✓ **ALWAYS save next BEFORE curr.next = prev**
- ✓ **Arrows show direction:** ──→ (forward), ←── (backward)
- ✓ **3 pointers needed:** prev, curr, next
- ✓ **Loop until curr == null**
- ✓ **After loop: prev is new head!**

🎨 Three Types of Problems

Problem Type	What Changes	Key Challenge
K-Group Reversal	Reverse in chunks of K	Reconnecting groups
Position Reversal	Reverse middle section	Keeping before/after intact
Rotation	Move tail to front	Breaking & reconnecting

2 How to Recognize These Problems

🎯 Pattern Recognition Guide

Quick Decision Tree

```
❓ WHICH PROBLEM IS IT?

┌───────────────────────────────────┐
│ Does problem mention "groups" or "k"? │
└───────────────────────────────────┘
                │
      YES ──────┴────── NO
       ↓                 ↓
  ┌─────────┐   ┌──────────────────────────┐
  │ K-GROUP │   │ Position indices given?  │
  └─────────┘   └──────────────────────────┘
                            │
               YES ─────────┴───────── NO
                ↓                       ↓
          ┌──────────┐           ┌────────┐
          │ POSITION │           │ ROTATE │
          │ REVERSAL │           │  LIST  │
          └──────────┘           └────────┘
```

3 Problem 1: Reverse Nodes in K-Group

Problem Statement - LeetCode 25

Reverse nodes in groups of **k**. If remaining nodes < k, leave them as is.

📊 ENHANCED Visual Walkthrough with Proper Arrows

```
🔍 COMPLETE EXAMPLE: k = 3

Input: 1 ──→ 2 ──→ 3 ──→ 4 ──→ 5 ──→ 6 ──→ 7 ──→ 8 ──→ null

╔═══════════════════════════════════════════════════╗
║ GROUP 1: Reverse nodes 1, 2, 3                    ║
╚═══════════════════════════════════════════════════╝

BEFORE REVERSAL:

left               right
 │                   │
 ↓                   ↓
┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
│ 1 │───→│ 2 │───→│ 3 │───→│ 4 │───→│ 5 │───→│ 6 │───→│ 7 │───→│ 8 │──→ null
└───┘    └───┘    └───┘    └───┘    └───┘    └───┘    └───┘    └───┘
                             ↑
                       nextLeft (save this!)

AFTER REVERSAL:

        ┌──────────┐
        ↓          │
┌───┐ ┌───┐ ┌───┐  │ ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
│ 3 │←─│ 2 │←─│ 1 ││──│ 4 │───→│ 5 │───→│ 6 │───→│ 7 │───→│ 8 │──→ null
└───┘ └───┘ └───┘│   └───┘    └───┘    └───┘    └───┘    └───┘
  ↑           ↑  │     ↑
right    prevLeft │    left (moved here)
(new HEAD)        │
                  └─ This connection needs to be made!

KEY CHANGES:
• Arrows within group now point backward (3←2←1)
• right (3) is now the front of this group
• left (1) is now the back of this group
• prevLeft will be 1 for next iteration
• nextLeft (4) is where we continue

╔═══════════════════════════════════════════════════╗
║ GROUP 2: Reverse nodes 4, 5, 6                    ║
╚═══════════════════════════════════════════════════╝

BEFORE REVERSAL:

                              prevLeft    left              right
                                  │         │                  │
                                  ↓         ↓                  ↓
┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
│ 3 │───→│ 2 │───→│ 1 │    │ 4 │───→│ 5 │───→│ 6 │───→│ 7 │───→│ 8 │──→ null
└───┘    └───┘    └───┘    └───┘    └───┘    └───┘    └───┘    └───┘
                                                         ↑
                                                  nextLeft (save!)

AFTER REVERSAL & RECONNECTION:

                            ┌──────────┐
                            ↓          │
┌───┐    ┌───┐    ┌───┐    ┌───┐ ┌───┐ ┌───┐│   ┌───┐    ┌───┐
│ 3 │───→│ 2 │───→│ 1 │───→│ 6 │←─│ 5 │←─│ 4 ││──│ 7 │───→│ 8 │──→ null
└───┘    └───┘    └───┘    └───┘ └───┘ └───┘│   └───┘    └───┘
                    ↑         ↑        ↑     │     ↑
                prevLeft    right   new tail │    left
                (old left)                   │
                                             └─ Connection point

CONNECTIONS MADE:
1. prevLeft (1) ──→ right (6)  [connects group 1 to group 2]
2. old left (4) becomes new prevLeft

╔═══════════════════════════════════════════════════╗
║ REMAINDER: Nodes 7, 8 (only 2 < k=3, DON'T REVERSE!) ║
╚═══════════════════════════════════════════════════╝

                                             prevLeft    left
                                                │          │
                                                ↓          ↓
┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
│ 3 │───→│ 2 │───→│ 1 │───→│ 6 │───→│ 5 │───→│ 4 │───→│ 7 │───→│ 8 │──→ null
└───┘    └───┘    └───┘    └───┘    └───┘    └───┘    └───┘    └───┘
                                                         ↑
                                          Connect prevLeft ─┘

✅ FINAL RESULT:
3 ──→ 2 ──→ 1 ──→ 6 ──→ 5 ──→ 4 ──→ 7 ──→ 8 ──→ null

GROUP STRUCTURE:
[3→2→1] (reversed) → [6→5→4] (reversed) → [7→8] (unchanged)
```

💻 Complete Java Implementation

```java
/**
 * Reverse Nodes in K-Group
 * LeetCode 25
 * Time: O(N), Space: O(1)
 */

class Solution {
    // Helper: Reverse k nodes starting from head
    private void reverse(ListNode head, int k) {
        ListNode curr = head;
        ListNode prev = null;
        ListNode next = null;

        while (k > 0) {
            next = curr.next;      // Save next
            curr.next = prev;      // Reverse arrow ──→ to ←──
            prev = curr;           // Move prev forward
            curr = next;           // Move curr forward
            k--;
        }
    }

    public ListNode reverseKGroup(ListNode head, int k) {
        if (head == null) return null;

        ListNode left = head;
        ListNode right = null;
        ListNode prevLeft = null;
        ListNode result = null;

        while (true) {
            right = left;

            // Find right (k-1 steps from left)
            for (int i = 0; i < k - 1; i++) {
                if (right == null) break;
                right = right.next;
            }

            // Not enough nodes? Connect and break
            if (right == null) {
                if (prevLeft != null) {
                    prevLeft.next = left;
                }
                break;
            }

            ListNode nextLeft = right.next;  // Save!
            reverse(left, k);               // Reverse group

            // Connect previous group to this group
            if (prevLeft != null) {
                prevLeft.next = right;
            }

            if (result == null) result = right;

            prevLeft = left;
            left = nextLeft;

            if (left == null) break;
        }

        return result == null ? head : result;
    }
}
```

💡 Key Takeaways:

- ✓ **Arrows reverse:** ──→ becomes ←── within group
- ✓ **After reversal:** right is new front, left is new back
- ✓ **prevLeft connects groups** together
- ✓ **Always save nextLeft** before reversing!

4 Problem 2: Reverse Between Positions

Problem - LeetCode 92

Reverse from position `left` to `right`

📊 Enhanced Visual with Proper Arrows - IMPROVED!

```
🔍 EXAMPLE: Reverse positions 2 to 4

══════════════════════════════════════════════
STEP 1: BEFORE REVERSAL
══════════════════════════════════════════════

 before          left                  right
   ↓               ↓                     ↓
┌────┐          ┌────┐          ┌────┐          ┌────┐          ┌────┐
│ 10 │─────────→│ 20 │─────────→│ 30 │─────────→│ 40 │─────────→│ 50 │──→ null
└────┘          └────┘          └────┘          └────┘          └────┘
 pos:1             2               3               4               5

══════════════════════════════════════════════
STEP 2: AFTER REVERSAL (Segment 20-30-40 is now 40-30-20)
══════════════════════════════════════════════

 before                                                          after
   ↓                                                               ↓
┌────┐          ┌────┐          ┌────┐          ┌────┐          ┌────┐
│ 10 │─┐        │ 40 │←─────────│ 30 │←─────────│ 20 │─┐        │ 50 │──→ null
└────┘ │        └────┘          └────┘          └────┘ │        └────┘
       │          ↑                                    │          ↑
       │          │                                    │          │
       └──────────┘                                    └──────────┘

    Connection 1:                                  Connection 2:
  before.next = 40                               old_left.next = 50
  (new head of                                  (tail of reversed
   reversed segment)                             segment to after)

══════════════════════════════════════════════
STEP 3: FINAL RESULT - Fully Reconnected
══════════════════════════════════════════════

┌────┐          ┌────┐          ┌────┐          ┌────┐          ┌────┐
│ 10 │─────────→│ 40 │─────────→│ 30 │─────────→│ 20 │─────────→│ 50 │──→ null
└────┘          └────┘          └────┘          └────┘          └────┘

Positions:  1              2               3               4               5

🔑 KEY RECONNECTION LOGIC:
══════════════════════════════════════════════
1. Reverse segment [20, 30, 40] internally → Result: 40 ← 30 ← 20

2. Make TWO critical connections:
   • Connection 1: before (10) ──→ new head (40)
   • Connection 2: old left (20) ──→ after (50)

3. These connections "stitch" the reversed segment back into the list!
```

💻 Java Code

```java
public ListNode reverseBetween(ListNode head, int left, int right) {
    if (head == null || left == right) return head;

    ListNode t = head;
    ListNode before = null;
    int pos = 1;

    // Find "before" node
    while (t != null && pos < left) {
        before = t;
        t = t.next;
        pos++;
    }

    // Reverse segment
    ListNode curr = t;
    ListNode prev = null;
    ListNode next = null;
    int count = right - left + 1;

    while (count > 0) {
        next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
        count--;
    }

    // Reconnect
    t.next = curr;  // old left → after

    if (before != null) {
        before.next = prev;  // before → new head
        return head;
    } else {
        return prev;  // left==1 case
    }
}
```

5 Problem 3: Rotate List

Problem - LeetCode 61

Rotate list to the **right** by k places

📊 Enhanced Rotation Visual

```
🔍 ROTATION WITH RING FORMATION

Input: 1 ──→ 2 ──→ 3 ──→ 4 ──→ 5 ──→ null, k = 2
N = 5, k = 2 % 5 = 2

╔════════════════════════════════════════════╗
║ STEP 1: Original List                      ║
╚════════════════════════════════════════════╝

┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
│ 1 │───→│ 2 │───→│ 3 │───→│ 4 │───→│ 5 │───→ null
└───┘    └───┘    └───┘    └───┘    └───┘
                    ↑                  ↑
                 new tail          last node
             (position N-k=3)

╔════════════════════════════════════════════╗
║ STEP 2: Create Ring (Connect last to first) ║
╚════════════════════════════════════════════╝

    ┌──────────────────────────────────┐
    │                                  ↓
    │  ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
    └──│ 1 │←───│ 2 │←───│ 3 │←───│ 4 │←───│ 5 │
       └───┘    └───┘    └───┘    └───┘    └───┘
                                    ↑
                                 new head
                             (position N-k+1=4)

╔════════════════════════════════════════════╗
║ STEP 3: Break Ring at New Tail             ║
╚════════════════════════════════════════════╝

                       new tail
                          │
                          ↓
       ┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
       │ 1 │───→│ 2 │───→│ 3 │ ╳ │ 4 │───→│ 5 │─┐
       └───┘    └───┘    └───┘    └───┘    └───┘ │
         ↑                          ↑             │
         │                       new head         │
         └────────────────────────────────────────┘

Set node 3's next = null

╔════════════════════════════════════════════╗
║ FINAL RESULT                               ║
╚════════════════════════════════════════════╝

┌───┐    ┌───┐    ┌───┐    ┌───┐    ┌───┐
│ 4 │───→│ 5 │───→│ 1 │───→│ 2 │───→│ 3 │───→ null
└───┘    └───┘    └───┘    └───┘    └───┘
  ↑
new HEAD

Result: 4 ──→ 5 ──→ 1 ──→ 2 ──→ 3 ──→ null
```

💻 Java Code

```java
public ListNode rotateRight(ListNode head, int k) {
    if (head == null) return null;

    // Find length and last node
    ListNode last = head;
    int n = 1;
    while (last.next != null) {
        last = last.next;
        n++;
    }

    // Optimize k
    k = k % n;
    if (k == 0) return head;

    // Find new tail at position (n-k)
    ListNode newTail = head;
    for (int i = 1; i < n - k; i++) {
        newTail = newTail.next;
    }

    // Perform rotation
    ListNode newHead = newTail.next;  // Save new head
    last.next = head;                 // Create ring
    newTail.next = null;              // Break ring

    return newHead;
}
```

6 Summary & Quick Reference

Arrow Legend

──→ Forward arrow (original direction)
←── Backward arrow (after reversal)
╳ Connection to be broken
│ ↓ ↑ Pointer indicators

Quick Reference

Problem	Key Technique	Critical Point
K-Group	Reverse in chunks	prevLeft connects groups
Position	Reverse middle only	Find "before" node first
Rotate	Ring formation	Break at (N-k) position

✅ Master Checklist

- ☐ Arrows show direction: ──→ and ←──
- ☐ Always save next before breaking connection
- ☐ Reversal changes arrow direction within segment
- ☐ After reversal: old start → new end, old end → new start

🎓 Mastered Linked List Reversal with Proper Arrows!

✓ Clear arrow directions: ──→ and ←──
✓ Visual node connections
✓ All reconnection logic illustrated

Ready to solve any reversal problem! 🚀